

Smart Automation & Monitoring Hub

Project documentation: ESP32, Wokwi, Blynk, MQTT (HiveMQ) and Node-RED

Author: Danish Nadeem

1. Project overview

The Smart Automation & Monitoring Hub is an ESP32 project that reads four sensors (DHT22 temperature and humidity, LDR light level, PIR motion and HC-SR04 distance) and controls two actuators automatically: a fan and a light. Each actuator is simulated by an LED with a 220 Ω resistor. The sensor data is sent to Blynk (web dashboard and mobile alerts) and published over MQTT to a Node-RED dashboard. Everything runs in the Wokwi simulator, so no physical hardware is needed.

Architecture

Part	Role
Wokwi (ESP32, sensors, LEDs)	Runs the firmware and the circuit. The ESP32 reaches the internet through the virtual WiFi network Wokwi-GUEST .
Blynk Cloud	Template, datastreams and events. Web dashboard with gauges, LEDs and charts, plus phone alerts.
HiveMQ public broker	broker.hivemq.com , port 1883 . The ESP32 publishes seven topics to it.
Node-RED	Subscribes to the MQTT topics and shows gauges at localhost:1880/ui .

Node-RED and Blynk are not connected to each other. Both receive their data directly from the ESP32.

Automation logic

Function	Rule
Fan (GPIO 26)	ON when temperature > 30.0 °C
Light (GPIO 25)	ON when the LDR reading > 3000 (dark) and motion is detected
Blynk event motion_alert	Sent when the PIR value changes from 0 to 1 (new motion)
Blynk event object_close	Sent when the distance is below 10 cm; it is sent once and re-armed when the distance is 10 cm or more
Loop interval	Sensors are read, relays set, and data sent every 2 seconds

Setup order

1. Install the tools (VS Code, PlatformIO IDE, Wokwi Simulator, Node.js).
2. Set up Blynk (template, datastreams, events, device).
3. Decide the MQTT broker and topics (nothing needs to be created).
4. Create the PlatformIO / Wokwi project files.
5. Paste the Blynk values into main.cpp, build, and start the simulator.
6. Check that the data reaches the broker (HiveMQ Websockets client).
7. Set up Node-RED (flow and dashboard).
8. Install the Blynk phone app (same account) for notifications.
9. Run the full test.

Tools to install (once)

- **VS Code.**
- In VS Code Extensions: **PlatformIO IDE** (restart VS Code afterwards) and **Wokwi Simulator**.
- Wokwi free license: press **Ctrl+Shift+P**, run **Wokwi: Request a Free License**, and activate it in the browser.
- **Node.js** (LTS) from nodejs.org, needed for Node-RED. Check with `node -v` in a Command Prompt.

2. Blynk setup

1. Log in at **blynk.cloud**. Go to **Developer Zone → My Templates → New Template**. Name: **Smart Automation Hub**, Hardware: **ESP32**, Connection: **WiFi**.
2. In the **Datastreams** tab create the seven Virtual Pin datastreams listed below.
3. In the **Events** tab create the two events listed below.
4. In the **Web Dashboard** tab add the widgets listed below.
5. Save the template. Then **Devices → New Device → From Template** and choose the template.
6. From the device page copy the **Template ID**, **Template Name** and **Auth Token**; they are pasted into `main.cpp` later.

Datastreams

Name	Pin	Type	Min	Max	Reason
Temperature	V0	Double	-40	80	DHT22 measuring range
Humidity	V1	Double	0	100	Percentage
Light Level	V2	Integer	0	4095	ESP32 ADC is 12-bit
Motion Status	V3	Integer	0	1	0 = no motion, 1 = motion
Distance	V4	Double	0	400	HC-SR04 range up to 400 cm
Fan Status	V5	Integer	0	1	0 = OFF, 1 = ON
Light Status	V6	Integer	0	1	0 = OFF, 1 = ON

Events

Code	Name	Type
<code>motion_alert</code>	Motion Detected	Info
<code>object_close</code>	Object Too Close	Warning

- **Condition:** remove the condition row with the X button and leave it empty. The ESP32 code triggers the events itself with `Blynk.logEvent()`, so Blynk does not need to watch a datastream.
- **Notifications:** Enable notifications ON (push).
- **Limits:** 1 notification per 1 minute.
- The Event Code must match the code exactly: lowercase, with an underscore, no spaces. Save the template after every change.

Web dashboard widgets

- Gauges: Temperature (V0), Humidity (V1), Light Level (V2), Distance (V4).
- LED widgets: Motion Status (V3), Fan Status (V5), Light Status (V6).
- Charts: Temperature, Humidity, Light Level and Distance (see the charts in Figures 3 and 4).

3. MQTT broker (HiveMQ public broker)

A free public broker is used, so nothing has to be created or installed and no account is needed.

Setting	Value
Broker	broker.hivemq.com
Port (ESP32 and Node-RED)	1883 (plain, unencrypted)
Username / password	none
Topic prefix	danish/automation/

Topics published by the ESP32 (values are sent with two decimals):

Topic	Content
danish/automation/temperature	Temperature in °C
danish/automation/humidity	Humidity in %
danish/automation/light	LDR reading (0 to 4095)
danish/automation/motion	Motion (0 or 1)
danish/automation/distance	Distance in cm
danish/automation/fan	Fan state (0 or 1)
danish/automation/lightstatus	Light state (0 or 1)

Because the broker is public, the prefix should be unique so that other people's data is not mixed with yours. If the prefix is changed it must be changed in main.cpp and in every Node-RED mqtt in node.

Checking the broker with the HiveMQ Websockets client

1. Open [hivemq.com/demos/websocket-client](https://broker.hivemq.com/demos/websocket-client) in a new tab.
2. Host: `broker.hivemq.com` (replace the default `mqtt-dashboard.com`). Port: 8884. SSL ticked. Username and Password empty. Click **Connect**; the dot turns green and shows *connected*.
3. Open **Subscriptions**, click **Add New Topic Subscription**, enter `danish/automation/#` and subscribe. The # catches all seven topics.
4. With the Wokwi simulator running, messages arrive in the **Messages** panel every 2 seconds. The **Publish** panel and its Retain option are not used.
 - If a *Socket error* appears, check the host and try port 8000 with SSL switched off; also try without a VPN or ad-blocker.
 - If the page says *Not connected* after being idle, click Connect again before subscribing.
 - Opening `broker.hivemq.com` in a normal browser tab shows HiveMQ's website and pricing page. The address is meant for MQTT clients; nothing has to be bought.

4. Project files and code

1. In VS Code open PlatformIO Home → **New Project**. Name: **smart-automation-hub**, Board: **Espressif ESP32 Dev Module**, Framework: **Arduino**.
2. Fill the project with the four files below.
3. In `src/main.cpp` paste the three Blynk values (quotes contain only the value, no extra spaces).
4. Click the PlatformIO **Build** (tick) button. **SUCCESS** must appear; libraries download on the first build.
5. Open `diagram.json`, press **Ctrl+Shift+P** and run **Wokwi: Start Simulator**.
6. The Serial monitor should show that Blynk connected, Connecting to MQTT...connected, and a data line every 2 seconds. Leave the simulator running.

File	Location	Content
<code>platformio.ini</code>	project root	Board and libraries (Blynk, DHT, PubSubClient)
<code>wokwi.toml</code>	project root	Firmware paths for Wokwi

File	Location	Content
diagram.json	project root	Circuit (section 5)
main.cpp	src folder	Complete firmware

main.cpp check	What it must match
WiFi: ssid = "Wokwi-GUEST", empty password	Wokwi virtual WiFi; do not change
mqttServer, mqttPort, topicPrefix	Section 3 and the Node-RED broker node
Blynk.logEvent("motion_alert" / "object_close")	Event Codes in Blynk
Blynk.virtualWrite(V0 ... V6)	Datastream pins in Blynk
#define pins	GPIO pins in diagram.json

platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
  blynkkk/Blynk@^1.3.2
  adafruit/DHT sensor library@^1.4.4
  knolleary/PubSubClient@^2.8
```

wokwi.toml

```
[wokwi]
version = 1
firmware = '.pio/build/esp32dev/firmware.bin'
elf = '.pio/build/esp32dev/firmware.elf'
```

src/main.cpp

```
#define BLYNK_TEMPLATE_ID "PASTE_YOUR_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "PASTE_YOUR_TEMPLATE_NAME"
#define BLYNK_AUTH_TOKEN "PASTE_YOUR_AUTH_TOKEN"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <PubSubClient.h>

char ssid[] = "Wokwi-GUEST";
char pass[] = "";

// ----- MQTT Settings -----
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* topicPrefix = "danish/automation/";

WiFiClient espClient;
PubSubClient mqttClient(espClient);

// ----- Pins -----
#define DHT_PIN 15
#define DHT_TYPE DHT22
#define LDR_PIN 34
#define PIR_PIN 27
#define TRIG_PIN 18
#define ECHO_PIN 19
#define FAN_RELAY 26
#define LIGHT_RELAY 25
#define DISTANCE_ALERT_THRESHOLD 10.0

DHT dht(DHT_PIN, DHT_TYPE);
BlynkTimer timer;

bool lastMotionState = false;
bool distanceAlertSent = false;

float readDistanceCM() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    return duration * 0.034 / 2;
}

void connectMQTT() {
    while (!mqttClient.connected()) {
        Serial.print("Connecting to MQTT...");
        String clientId = "ESP32-Client-" + String(random(0xffff), HEX);
        if (mqttClient.connect(clientId.c_str())) {
            Serial.println("connected");
        } else {
            Serial.print("failed, rc=");
            Serial.print(mqttClient.state());
            Serial.println(" retrying in 2s");
            delay(2000);
        }
    }
}

void publishMQTT(const char* subtopic, float value) {
    char topic[60];
    char payload[20];
    snprintf(topic, sizeof(topic), "%s%s", topicPrefix, subtopic);
    snprintf(payload, sizeof(payload), "%.2f", value);
    mqttClient.publish(topic, payload);
}

void runAutomationAndPublish() {
    // ---- Read Sensors ----
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();
    int lightLevel = analogRead(LDR_PIN);
    bool motionDetected = digitalRead(PIR_PIN);
    float distance = readDistanceCM();

    if (isnan(temperature) || isnan(humidity)) {
        Serial.println("Failed to read from DHT sensor!");
        return;
    }

    // ---- Automation Logic ----
```

```

bool fanShouldBeOn = (temperature > 30.0);
bool lightShouldBeOn = (lightLevel > 3000) && motionDetected; // dark + motion

digitalWrite(FAN_RELAY, fanShouldBeOn ? HIGH : LOW);
digitalWrite(LIGHT_RELAY, lightShouldBeOn ? HIGH : LOW);

// ---- Send to Blynk ----
Blynk.virtualWrite(V0, temperature);
Blynk.virtualWrite(V1, humidity);
Blynk.virtualWrite(V2, lightLevel);
Blynk.virtualWrite(V3, motionDetected ? 1 : 0);
Blynk.virtualWrite(V4, distance);
Blynk.virtualWrite(V5, fanShouldBeOn ? 1 : 0);
Blynk.virtualWrite(V6, lightShouldBeOn ? 1 : 0);

if (motionDetected && !lastMotionState) {
  Blynk.logEvent("motion_alert", "Motion detected near the sensor!");
}
lastMotionState = motionDetected;

if (distance > 0 && distance < DISTANCE_ALERT_THRESHOLD && !distanceAlertSent) {
  Blynk.logEvent("object_close", "Object detected too close!");
  distanceAlertSent = true;
} else if (distance >= DISTANCE_ALERT_THRESHOLD) {
  distanceAlertSent = false;
}

// ---- Publish to MQTT (Node-RED dashboard) ----
if (!mqttClient.connected()) {
  connectMQTT();
}
publishMQTT("temperature", temperature);
publishMQTT("humidity", humidity);
publishMQTT("light", lightLevel);
publishMQTT("motion", motionDetected ? 1 : 0);
publishMQTT("distance", distance);
publishMQTT("fan", fanShouldBeOn ? 1 : 0);
publishMQTT("lightstatus", lightShouldBeOn ? 1 : 0);

// ---- Debug ----
Serial.print("Temp: "); Serial.print(temperature);
Serial.print(" | Humidity: "); Serial.print(humidity);
Serial.print(" | Light: "); Serial.print(lightLevel);
Serial.print(" | Motion: "); Serial.print(motionDetected);
Serial.print(" | Distance: "); Serial.print(distance);
Serial.print(" | Fan: "); Serial.print(fanShouldBeOn);
Serial.print(" | LightRelay: "); Serial.println(lightShouldBeOn);
}

void setup() {
  Serial.begin(115200);
  dht.begin();

  pinMode(PIR_PIN, INPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(FAN_RELAY, OUTPUT);
  pinMode(LIGHT_RELAY, OUTPUT);

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  mqttClient.setServer(mqttServer, mqttPort);
  connectMQTT();

  timer.setInterval(2000L, runAutomationAndPublish);
}

void loop() {
  Blynk.run();
  mqttClient.loop();
  timer.run();
}

```

Parts of the code

Part	What it does
PubSubClient mqttClient(espClient)	Creates the MQTT connection object on top of the WiFi client
connectMQTT()	Connects to the broker and retries if the connection is lost (random client ID)
publishMQTT()	Sends one value to its own topic, for example danish/automation/temperature
fanShouldBeOn, lightShouldBeOn	Automation decisions made from the sensor data; no button is needed
digitalWrite(FAN_RELAY, ...)	Switches the relay (LED) ON or OFF according to the decision

5. Circuit and wiring (Wokwi)

The circuit is defined in `diagram.json`. Notes: the LDR analog pin is **A0** (`ldr1:A0`); only GND.1, GND.2 and GND.3 are used, and several wires may share one GND pin. If parts overlap in Wokwi, drag them apart; the wires follow.

diagram.json

```
{
  "version": 1,
  "author": "Danish",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-dht22", "id": "dht1", "top": -258.9, "left": -149.4, "rotate": 270, "attrs": {} },
    { "type": "wokwi-photoresistor-sensor", "id": "ldr1", "top": -169.6, "left": 240.8, "rotate": 180, "attrs": {} },
    { "type": "wokwi-pir-motion-sensor", "id": "pir1", "top": -72.8, "left": -228.18, "rotate": 270, "attrs": {} },
    { "type": "wokwi-hc-sr04", "id": "ultrasonic1", "top": 33.9, "left": -181.5, "rotate": 90, "attrs": {} },
    { "type": "wokwi-led", "id": "fan_led", "top": 90, "left": 220, "attrs": { "color": "blue" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 90, "left": 280, "attrs": { "value": "220" } },
    { "type": "wokwi-led", "id": "light_led", "top": 150, "left": 220, "attrs": { "color": "yellow" } },
    { "type": "wokwi-resistor", "id": "r2", "top": 150, "left": 280, "attrs": { "value": "220" } }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],

    [ "dht1:VCC", "esp:3V3", "red", [] ],
    [ "dht1:SDA", "esp:15", "green", [] ],
    [ "dht1:GND", "esp:GND.1", "black", [] ],

    [ "ldr1:VCC", "esp:3V3", "red", [] ],
    [ "ldr1:A0", "esp:34", "yellow", [] ],
    [ "ldr1:GND", "esp:GND.2", "black", [] ],

    [ "pir1:VCC", "esp:5V", "red", [] ],
    [ "pir1:OUT", "esp:27", "orange", [] ],
    [ "pir1:GND", "esp:GND.3", "black", [] ],

    [ "ultrasonic1:VCC", "esp:5V", "red", [] ],
    [ "ultrasonic1:TRIG", "esp:18", "blue", [] ],
    [ "ultrasonic1:ECHO", "esp:19", "purple", [] ],
    [ "ultrasonic1:GND", "esp:GND.1", "black", [] ],

    [ "esp:26", "r1:1", "blue", [] ],
    [ "r1:2", "fan_led:A", "blue", [] ],
    [ "fan_led:C", "esp:GND.2", "black", [] ],

    [ "esp:25", "r2:1", "yellow", [] ],
    [ "r2:2", "light_led:A", "yellow", [] ],
    [ "light_led:C", "esp:GND.3", "black", [] ]
  ],
  "dependencies": {}
}
```

Sensors

Component	Component pin	ESP32 pin	Wire colour
DHT22	VCC	3V3	red
	SDA	GPIO 15	green
	GND	GND.1	black
LDR module	VCC	3V3	red
	AO	GPIO 34	yellow
	GND	GND.2	black
PIR sensor	VCC	5V	red
	OUT	GPIO 27	orange
	GND	GND.3	black
HC-SR04	VCC	5V	red
	TRIG	GPIO 18	blue
	ECHO	GPIO 19	purple
	GND	GND.1	black

LEDs (relay simulation)

LED	Connection
Fan LED (blue)	GPIO 26 → resistor r1 (220 Ω) pin 1; r1 pin 2 → LED anode (A); LED cathode (C) → GND.2
Light LED (yellow)	GPIO 25 → resistor r2 (220 Ω) pin 1; r2 pin 2 → LED anode (A); LED cathode (C) → GND.3

Pin summary

Component	GPIO
DHT22	GPIO 15
LDR	GPIO 34
PIR	GPIO 27
HC-SR04 TRIG	GPIO 18
HC-SR04 ECHO	GPIO 19
Fan relay (LED)	GPIO 26
Light relay (LED)	GPIO 25

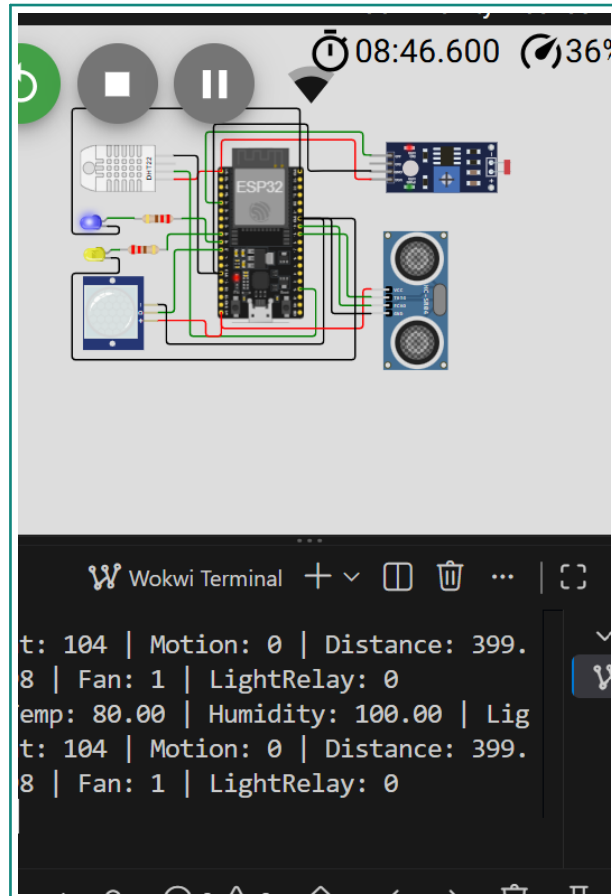


Figure 1. Wokwi simulator with the Wokwi Terminal (Serial monitor). Each line has the format Temp | Humidity | Light | Motion | Distance | Fan | LightRelay. This capture shows Temp 80.00, Humidity 100.00, Light 104, Motion 0, Distance about 399, Fan 1, LightRelay 0 (the DHT22 sliders are at their maximum, so the fan is ON).

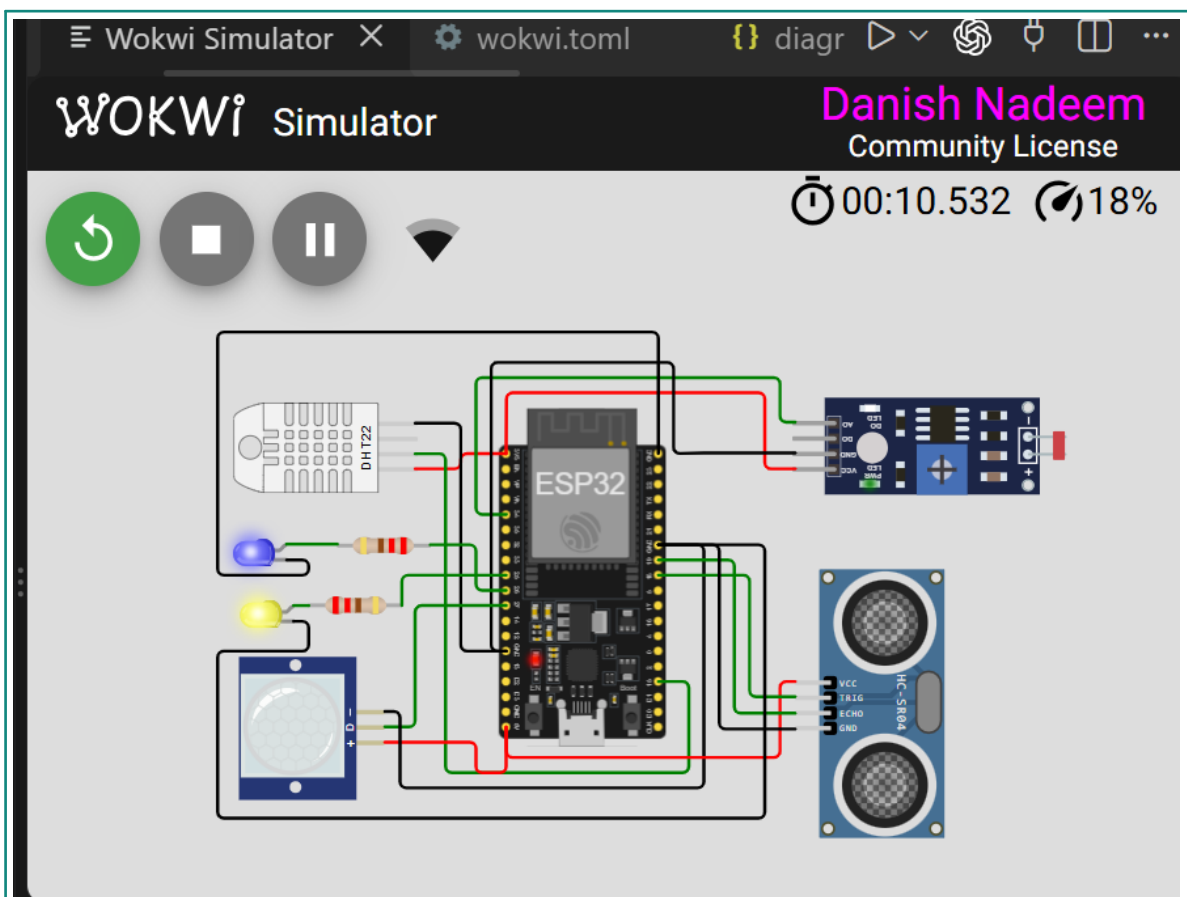


Figure 2. Completed circuit in the Wokwi simulator: ESP32 with the DHT22, LDR module, PIR sensor and HC-SR04, plus the blue (fan) and yellow (light) LEDs, each through a 220 Ω resistor.

6. Blynk dashboard and phone app

The Blynk web dashboard shows the gauges, LED widgets and charts built in section 2. For phone alerts install the **Blynk IoT** app, log in with the same account used on blynk.cloud, and allow notifications. Events can also be checked in the device **Timeline** in the Blynk console.

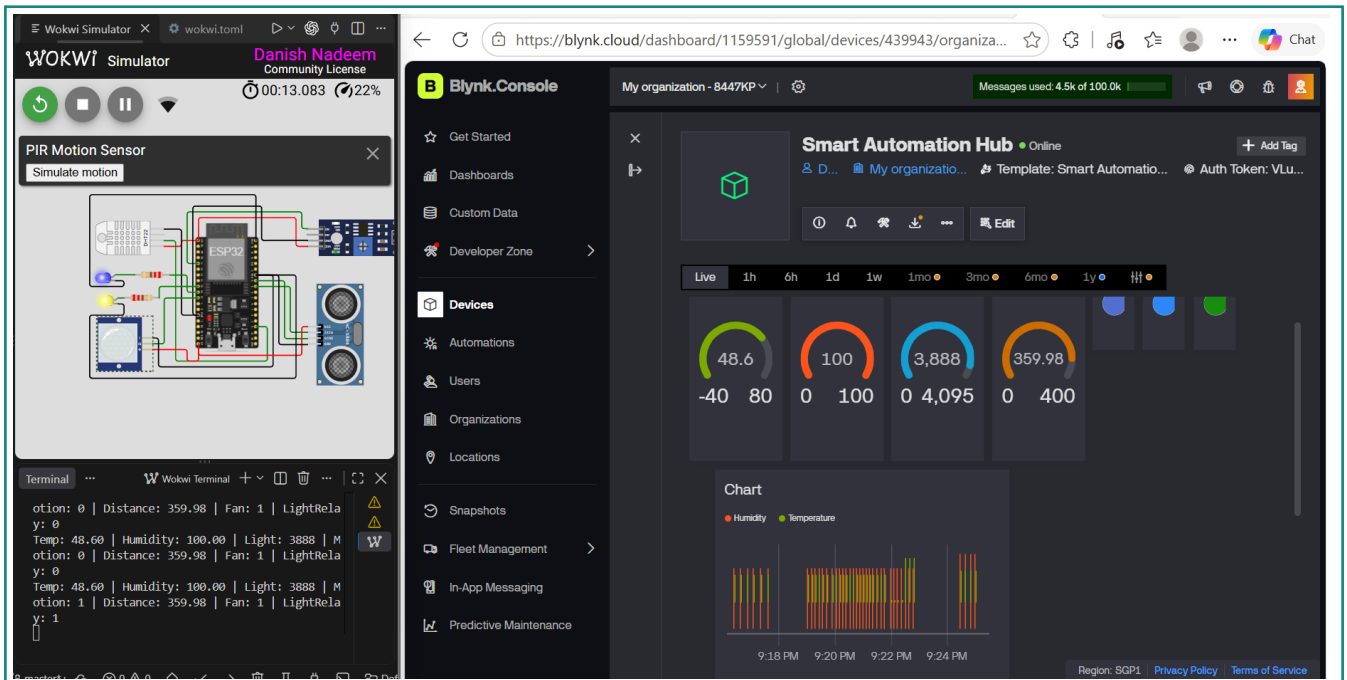


Figure 3. Wokwi simulator (left) and the Blynk console (right). The device Smart Automation Hub is Online. The gauges show Temperature 48.6, Humidity 100, Light Level 3,888 and Distance 359.98, matching the Serial monitor. The chart shows Humidity and Temperature.

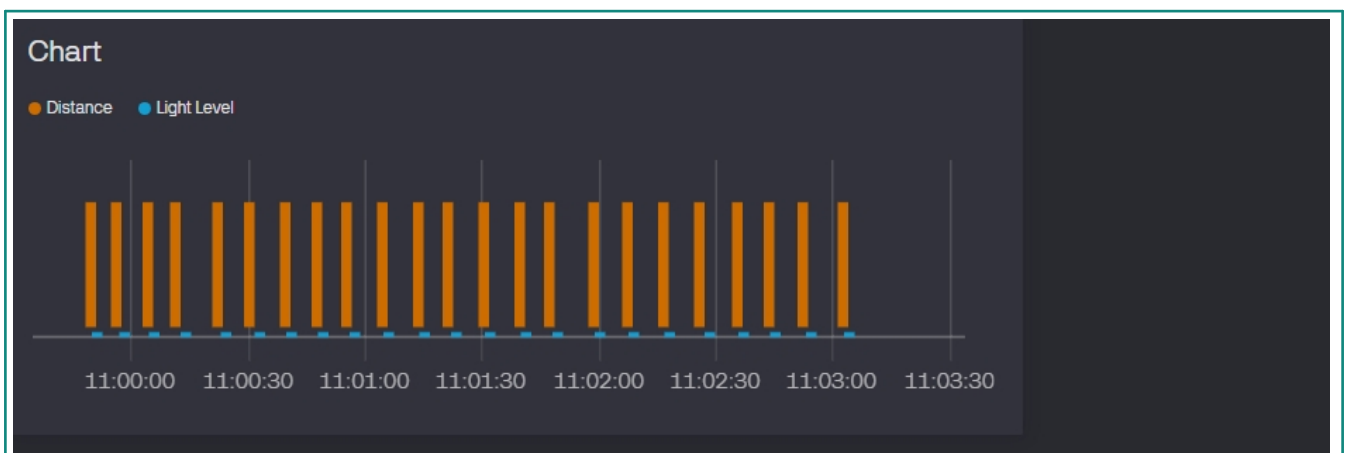


Figure 4. Blynk chart widget showing Distance and Light Level.

7. Node-RED dashboard

1. Install Node-RED: run `npm install -g --unsafe-perm node-red`, then `node-red`.
2. Open `localhost:1880` in the browser.
3. Menu → **Manage Palette** → **Install** and install **node-red-dashboard**.
4. Build the flow: an **mqtt in** node for each topic, wired to a dashboard widget (table below). Create the broker server once (`broker.hivemq.com`, port 1883) and select it in the other nodes. Output: auto-detect.
5. Click **Deploy**. Each mqtt in node must show a green dot with *connected*.
6. Open `localhost:1880/ui` (tab **Smart Hub**). The widgets update every 2 seconds while Wokwi is running.

mqtt in topic	Dashboard widget	Range
danish/automation/temperature	Gauge "Temperature" (°C)	-40 to 80
danish/automation/humidity	Gauge "Humidity" (%)	0 to 100
danish/automation/light	Gauge "Light Level"	0 to 4095
danish/automation/distance	Gauge "Distance" (cm)	0 to 400
danish/automation/motion	Text "Motion (0/1)"	0 or 1
danish/automation/fan	Text "Fan (0/1)"	0 or 1
danish/automation/lightstatus	Text "Light (0/1)"	0 or 1

FlowFuse (the browser-based Node-RED) uses Dashboard 2.0, so its dashboard nodes are different from `node-red-dashboard`.

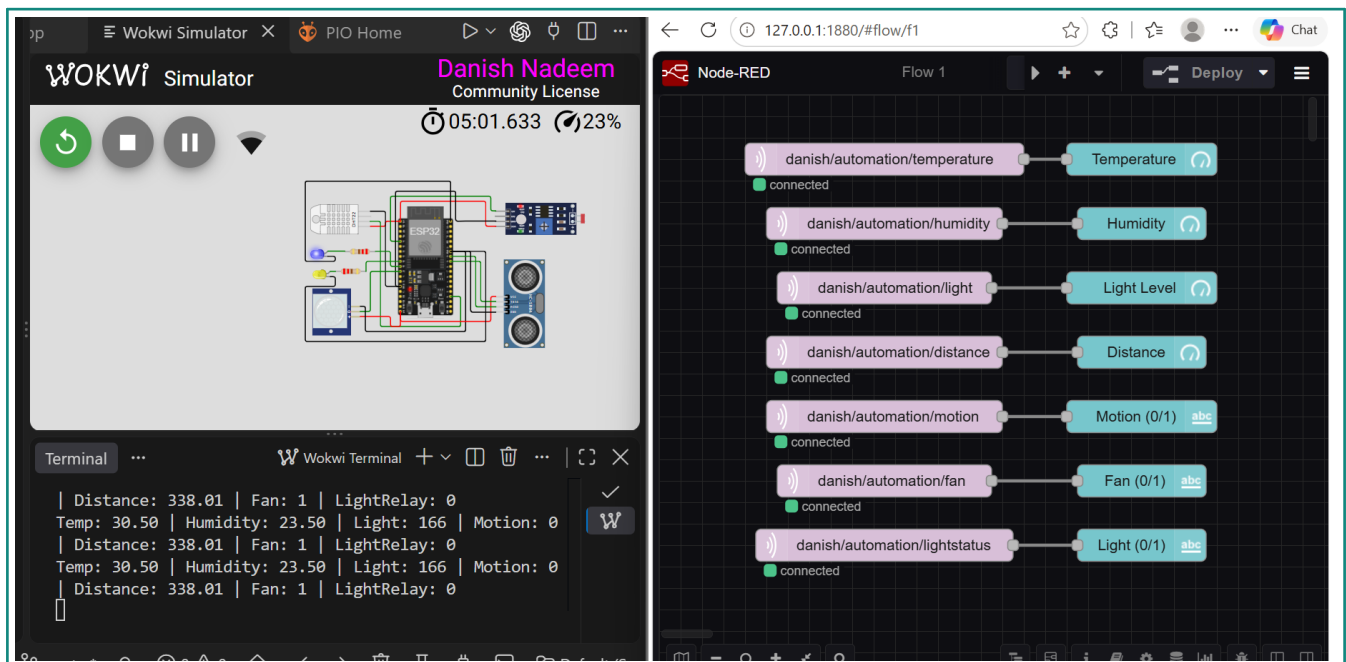


Figure 5. Wokwi simulator (left) and the Node-RED flow editor (right). Seven mqtt in nodes (temperature, humidity, light, distance, motion, fan, lightstatus) show connected and are wired to the dashboard widgets. Serial monitor: Temp 30.50, Humidity 23.50, Light 166, Motion 0, Distance 338.01, Fan 1, LightRelay 0.

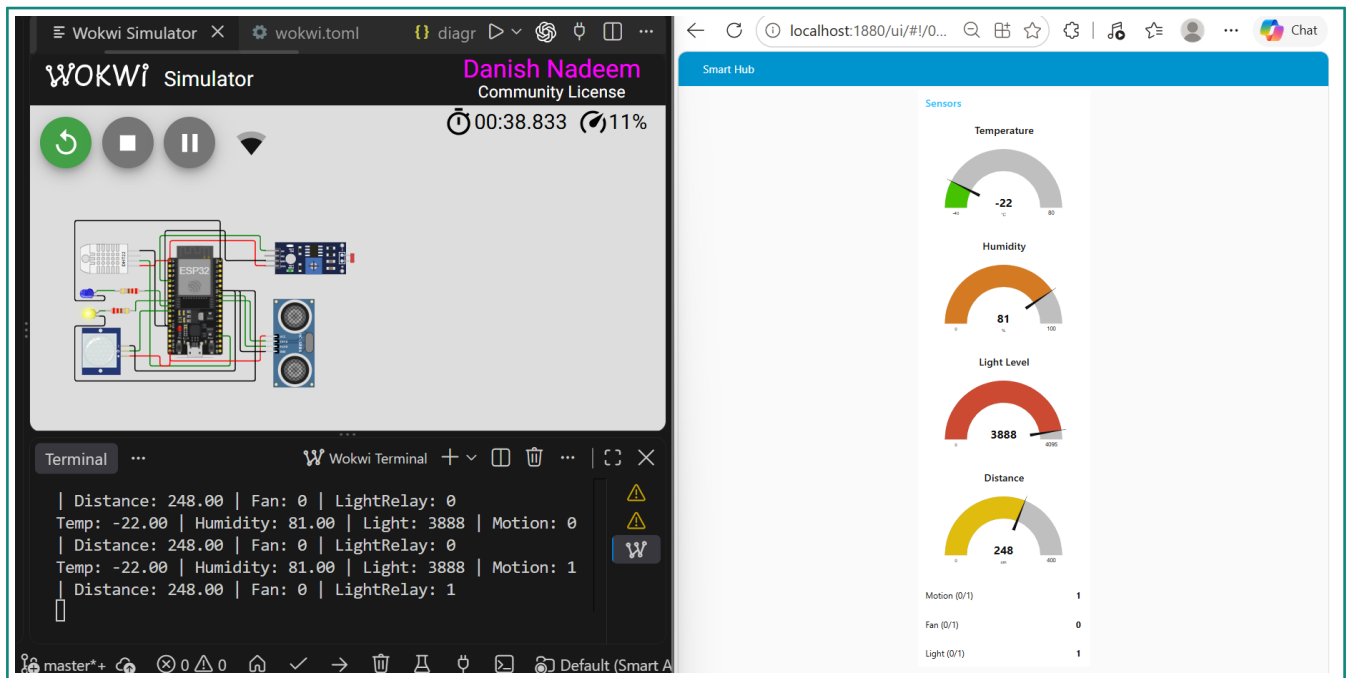


Figure 6. Node-RED dashboard (localhost:1880/ui, tab Smart Hub) next to the Wokwi Serial monitor. Temperature -22, Humidity 81, Light Level 3888, Distance 248, Motion 1, Fan 0, Light 1 match the Serial values (Motion 1, LightRelay 1).

8. Testing the whole system

Keep the Wokwi simulator running while testing.

Test	What to do	Expected result
Fan	Click the DHT22 and set the temperature above 30 °C	Blue LED ON, Fan: 1
Light	Click the LDR and make it dark (reading above 3000), then trigger the PIR (<i>Simulate motion</i>)	Yellow LED ON, LightRelay: 1
Motion alert	Trigger the PIR	Blynk notification "Motion Detected"
Object close	Set the HC-SR04 distance below 10 cm	Blynk notification "Object Too Close"
Node-RED	Open localhost:1880/ui	Gauges and texts update
Blynk	Open the Blynk web dashboard	Gauges, LEDs and charts update

In Wokwi the LDR reading becomes **higher when it is darker** (bright light gives a low reading such as 104; the dark readings in the screenshots are 3888). The light LED therefore needs a dark LDR **and** motion at the same time. The PIR output is only high for a few seconds and the code checks every 2 seconds, so trigger the motion again if *Motion: 1* does not appear in the Serial monitor.

9. Troubleshooting

Problem	What to check
Build error	Read the error text in PlatformIO; check platformio.ini and the library names.
Blynk does not connect	Copy Template ID, Template Name and Auth Token again; no extra spaces inside the quotes.
Blynk log: <i>Event 'motion_alert' (or 'object_close') not found in device template</i>	In the template Events tab the Event Codes must be exactly <code>motion_alert</code> and <code>object_close</code> (Blynk creates a code from the name by default). Save the template, and make sure the device uses that template (the Template ID matches <code>main.cpp</code>). Restart the simulator and test again.
MQTT failed, <code>rc=-2</code>	The ESP32 could not reach the broker. Let WiFi/Blynk connect first and check the internet and port 1883.
HiveMQ client: <i>Socket error</i>	Host <code>broker.hivemq.com</code> , port 8884 with SSL; or port 8000 without SSL. Switch off VPN/ad-blocker or try another browser.
HiveMQ client: <i>Not connected on Subscribe</i>	Connect again first (green dot), then subscribe.
No messages in the client	The Wokwi simulator must be running; topic must be <code>danish/automation/#</code> ; check the prefix spelling.
Node-RED gauge empty	Topic spelling must match exactly (case-sensitive), click Deploy, the <code>mqtt</code> in node must show connected, and Wokwi must be running.
Yellow LED never turns ON	Needs Light above 3000 and Motion 1 together; see the note in section 8.
Wire not connected in Wokwi	Check the pin name (LDR uses AO) and drag overlapping parts apart.